

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO

Estrategia de Articulación EnContexto · Departamento de Sistemas Digitales

Trabajo de consulta en parejas

SolarDB Pascual

Gobernanza de datos, automatización ETL e IoT sobre un repositorio relacional

Curso	Bases de Datos I (SD1006)
Grupo / Semestre	811 · 2026-II
Docente	Ramiro Grisales Montoya

Integrante 1 (nombre completo): Emmanuel Echeverri Quintero 1025894006

Fecha de entrega:

Enlace al repositorio en GitHub:

<https://github.com/emmanuelecheverriq/solardb-echeverri-quintero>

Parte A · Consulta

Cada respuesta está redactada con palabras propias y aplicada al caso SolarDB. Las referencias entre corchetes, p. ej. [3], remiten a la lista final. Las tablas y fragmentos de SQL no cuentan en el límite de 150 palabras.

Bloque 1 · Gobernanza de datos

G1 · Data governance frente a data management y data quality

Data governance es el ejercicio de autoridad, control y decisión compartida sobre cómo se gestionan los activos de datos [2]: define quién decide, bajo qué reglas y quién responde. Data management es la ejecución cotidiana, es decir, los procesos y herramientas que crean, mueven, almacenan y protegen esos datos [1]. Data quality es una de las áreas de esa gestión y se enfoca en medir y mejorar qué tan aptos son los datos para su uso [1][3]. DAMA coloca la gobernanza en el centro de su rueda, orientando a las demás áreas [1].

En SolarDB, la gobernanza decide que la irradiancia válida va de 0 a 1500 W/m² y quién puede cargar datos; la gestión es el pipeline ETL con PostgreSQL; la calidad es comprobar, con CHECK y conteo de filas rechazadas, que esa regla realmente se cumpla.

G2 · Roles: data owner, data steward y data custodian

El **data owner** responde por un conjunto de datos: autoriza accesos y aprueba las reglas de calidad. El **data steward** vigila el día a día: mantiene definiciones, metadatos y controles de calidad. El **data custodian** opera lo técnico: almacenamiento, copias de respaldo y aplicación de permisos [1].

Propuesta para SolarDB: owner, el docente responsable del proyecto (Departamento de Sistemas Digitales), porque la institución es dueña de la planta y de sus datos. Steward, el estudiante desarrollador del proyecto, quien mantiene el diccionario y las reglas de calidad. Custodian, quien administre el servidor PostgreSQL con el rol solar_admin, preferiblemente una persona distinta del owner. Separar los roles evita que una misma persona defina, modifique y audite sus propios datos.

G3 · Metadatos y catálogo de datos

Los metadatos son datos sobre los datos: nombre, tipo, significado, unidad, origen y responsable de cada elemento. Un catálogo de datos es el inventario central y consultable de esos metadatos, de modo que cualquiera encuentre un dato y sepa si puede confiar en él.

El diccionario de datos del Taller 1 documenta cada tabla y columna con su tipo, nulabilidad y descripción, pero no registra quién responde por el dato, de dónde proviene, qué regla de calidad lo valida ni cuándo fue revisado. Para operar como catálogo gobernado le faltan las cuatro columnas siguientes, además de la unidad de ingeniería y la clasificación de sensibilidad.

Columna a agregar	Qué registra	Ejemplo (tabla lectura, columna irradiancia)
responsable	Steward y owner del dato	Steward: desarrollador del proyecto · Owner: docente del proyecto
origen	De dónde proviene	Piranómetro vía gateway MQTT; campo irradiancia del payload
regla_calidad	Condición de validez o completitud	CHECK entre 0 y 1500 W/m ² ; no nulo en horas de sol
fecha_actualizacion	Última revisión del registro	2026-10-05 (con versión del diccionario)

G4 · Dimensiones de calidad y restricciones en PostgreSQL

Compleitud: están todos los datos requeridos. **Validez:** el valor respeta formato y dominio. **Unicidad:** una misma realidad no se registra dos veces. **Consistencia:** los datos no se contradicen entre tablas. **Oportunidad:** llegan a tiempo para usarse (p. ej., la lectura queda en la base minutos después de su ts). **Exactitud:** reflejan el valor real, lo que depende de la calibración del sensor y no se garantiza con una restricción SQL. Cuatro de ellas se traducen en las restricciones siguientes:

```
-- Compleitud: la marca de tiempo es obligatoria
ALTER TABLE lectura_demo ALTER COLUMN timestamps SET NOT NULL;
-- Validez: dominio físico y protocolos permitidos
ALTER TABLE lectura_demo ADD CONSTRAINT ck_irr CHECK (irradiancia BETWEEN 0 AND 1500);
ALTER TABLE dispositivo ADD CONSTRAINT ck_proto
CHECK (protocolo IN ('modbus','mqtt','opcua'));
-- Unicidad: un número de serie no se repite entre dispositivos
ALTER TABLE dispositivo ADD CONSTRAINT uq_serial UNIQUE (serial);
-- (en lectura_demo la PK (dispositivo_id, timestamps) ya evita lecturas repetidas)
-- Consistencia: toda lectura pertenece a un dispositivo existente
ALTER TABLE lectura ADD CONSTRAINT fk_lect_disp
FOREIGN KEY (dispositivo_id) REFERENCES dispositivo(dispositivo_id);
```

G5 · Mínimo privilegio y roles de SolarDB

El principio de mínimo privilegio (least privilege) establece que cada persona o proceso reciba solo los permisos indispensables para su tarea. Así, un error o una cuenta comprometida causa el menor daño posible. En PostgreSQL se aplica creando roles con CREATE ROLE, otorgando lo necesario con GRANT y retirando lo sobrante con REVOKE.

Para SolarDB se proponen tres roles: solar_ingesta (lo usa el ETL: escribe, no altera estructura), solar_lector (consulta, solo lectura) y solar_admin (administración). El script completo está en sql/03_roles.sql.

```
CREATE ROLE solar_lector LOGIN PASSWORD :pw_lector; -- clave por variable de psql
CREATE ROLE solar_ingesta LOGIN PASSWORD :pw_ingesta;
CREATE ROLE solar_admin LOGIN PASSWORD :pw_admin;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
GRANT SELECT ON lectura_demo TO solar_lector;
GRANT SELECT, INSERT ON lectura_demo TO solar_ingesta;
GRANT SELECT, INSERT, DELETE ON stg_lectura_raw TO solar_ingesta;
GRANT SELECT, INSERT, UPDATE ON etl_log TO solar_ingesta;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO solar_admin;
```

Tabla	solar_ingesta	solar_lector	solar_admin
lectura_demo (y lectura oficial)	SELECT, INSERT	SELECT	Todos
stg_lectura_raw	SELECT, INSERT, DELETE	Ninguno	Todos
etl_log	SELECT, INSERT, UPDATE	Ninguno	Todos
dispositivo (modelo oficial)	SELECT	SELECT	Todos

G6 · Ley 1581 de 2012 y postura ética

La ley define dato personal como cualquier información vinculada o asociable a una o varias personas naturales determinadas o determinables [4]. La telemetría de la planta (potencia, irradiancia, temperatura,

voltaje) describe equipos, no personas; por eso, en principio, no es dato personal. La excepción sería que algún valor permita identificar a un titular, por ejemplo el consumo de una vivienda concreta.

Si SolarDB registrara a los operadores que atienden alarmas, nombre, cédula y hora de atención sí serían datos personales y exigirían autorización previa, expresa e informada [4], finalidad declarada, acceso restringido y atención de derechos de consulta y rectificación.

Postura ética: recolectar lo mínimo, usar un código interno en vez de la cédula, informar la finalidad y no publicar datos reales en el repositorio de GitHub.

Bloque 2 · Automatización de procesos ETL

E1 · ETL frente a ELT

En ETL los datos se transforman antes de entrar al destino; en ELT primero se cargan crudos y se transforman dentro del propio motor de base de datos.

SolarDB se parece más a **ELT**: el mensaje del gateway se guarda intacto en `stg_lectura_raw.payload` (JSONB) y luego una sentencia SQL extrae atributos, convierte tipos y valida rangos dentro de PostgreSQL. Conservar el dato crudo permite reprocesar si cambia una regla y auditar el origen de cada fila. No es ELT puro, porque antes de cargar el script de Python solo comprueba que cada línea sea JSON válido, y porque el destino final (`lectura_demo`) es una tabla tipada con restricciones.

E2 · Carga completa, incremental e idempotencia

La **carga completa** reescribe todo el destino en cada corrida: es simple, pero costosa cuando los datos crecen. La **carga incremental** procesa solo lo nuevo o modificado, por ejemplo las lecturas con `ts` posterior a la última cargada. Un proceso es **idempotente** si ejecutarlo varias veces con los mismos datos deja el mismo resultado que ejecutarlo una vez.

`INSERT ... ON CONFLICT (dispositivo_id, ts) DO NOTHING` logra eso: cuando la clave primaria ya existe, PostgreSQL omite la fila sin error [5]. Así, reintentar tras una falla o recibir archivos solapados no duplica lecturas. Con `DO UPDATE` se obtendría un upsert que además corrige valores [5].

E3 · Formas de programar una carga

Criterio	cron / Programador de tareas	pg_cron	Apache Airflow
Qué es	Planificador del sistema operativo que lanza un comando en horarios fijos.	Extensión de PostgreSQL con sintaxis cron que ejecuta comandos SQL dentro de la base [6].	Plataforma de orquestación: los flujos se definen como DAG en Python, con planificación y monitoreo [7][8].
Cuándo conviene	Un script o comando simple y periódico, sin dependencias entre pasos.	Tareas que son SQL puro en el servidor (mantenimiento, agregaciones, llamar un procedimiento).	Muchos pasos con dependencias, reintentos automáticos, reprocesos históricos y alertas [8].
Complejidad	Baja: una línea de configuración.	Media: requiere cargar la extensión en <code>shared_preload_libraries</code> y reiniciar el servidor [6].	Alta: necesita instalar scheduler, base de metadatos y servidor web.

Decisión para este proyecto: cron en Linux o el Programador de tareas en Windows. El flujo es un solo comando (`python etl/run_etl.py`) que además usa Python, algo que `pg_cron` no ejecuta, y Airflow sería

excesivo para un pipeline de pocos pasos. Si en la Fase 1 aparecen varios flujos dependientes, Airflow sería el siguiente paso.

E4 · Bitácora del ETL y trazabilidad

Cada ejecución debe dejar una fila en `etl_log` con: identificador de ejecución, nombre del proceso, hora de inicio y fin, filas leídas, cargadas y rechazadas, estado (por ejemplo, EXITOSO o ERROR) y mensaje de error.

Es una herramienta de gobernanza porque permite la **trazabilidad (data lineage)**: responder de dónde vino cada dato, cuándo entró y con qué resultado. También sirve para auditar, para detectar degradación de calidad (si las rechazadas suben de un día a otro, algo falla en el sensor o en el gateway) y para demostrar que los controles se ejecutaron. Sin bitácora, un número incorrecto en la base no tiene un camino de regreso hacia su origen.

Bloque 3 · Internet de las Cosas (IoT)

I1 · Arquitectura IoT por capas

Dispositivo: el inversor (potencia AC, voltaje DC), el medidor (energía) y el piranómetro (irradiancia) generan las mediciones. **Gateway (edge):** lee los equipos por Modbus u OPC UA, normaliza las lecturas a JSON y las publica por MQTT; puede almacenar temporalmente si se cae la red. **Red o broker:** el broker MQTT distribuye los mensajes por topics. **Almacenamiento:** un suscriptor o el ETL guarda el mensaje crudo en `stg_lectura_raw` y luego lo pasa a `lectura`; **SolarDB es esta capa.** **Aplicación:** consultas SQL y reportes de datos (sin tableros, según el alcance).

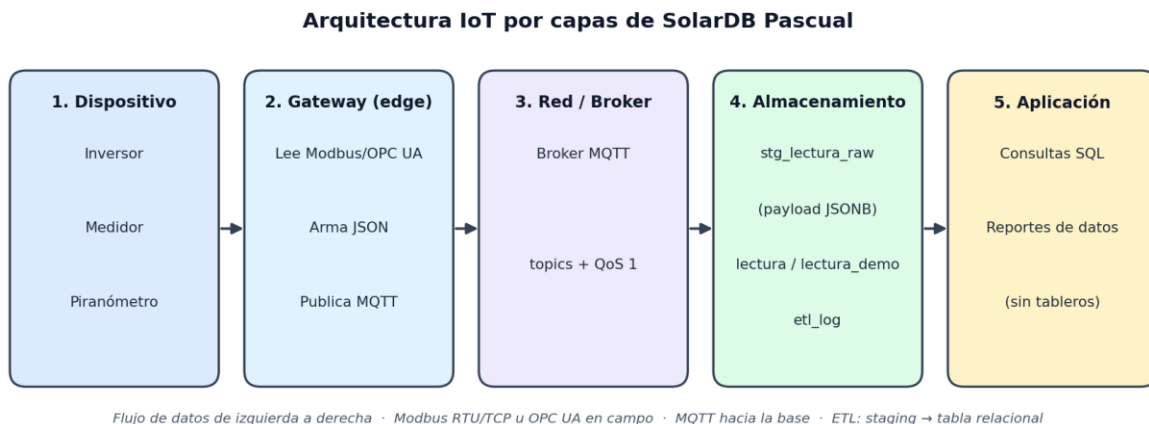


Figura 1. Arquitectura IoT por capas de SolarDB Pascual (elaboración propia).

I2 · MQTT: publish/subscribe, broker, topic y QoS

En MQTT los dispositivos **publican** mensajes en un **broker** y otros clientes se **suscriben** para recibirlos; unos y otros no se conocen entre sí [9]. El **topic** es una cadena jerárquica separada por «/» que el broker usa para enrutar [9]. La **QoS** fija la garantía de entrega: **0**, como máximo una vez y sin confirmación; **1**, al menos una vez, con posibles duplicados; **2**, exactamente una vez, mediante un intercambio de cuatro pasos [9].

Para SolarDB convendría QoS 1 en la telemetría: no se pierden datos y los duplicados los absorbe la clave primaria con `ON CONFLICT`.

```

pascualbravo/solar/<sitio>/<dispositivo>/<variable>
pascualbravo/solar/cubierta-bloque-a/inversor-01/p_ac
pascualbravo/solar/cubierta-bloque-a/inversor-01/alarma
pascualbravo/solar/cubierta-bloque-a/piranometro-01/irradiancia
pascualbravo/solar/cubierta-bloque-a/medidor-01/energia_kwh
-- suscribirse a todo un sitio:      pascualbravo/solar/cubierta-bloque-a/#
-- una variable en todos los sitios: pascualbravo/solar/+/+/alarma

```

13 · Modbus, MQTT y OPC UA

Criterio	Modbus (RTU/TCP)	MQTT	OPC UA
Modelo de comunicación	Cliente/servidor (maestro/esclavo); RTU por línea serial, TCP por Ethernet [10].	Publicación/suscripción mediante un broker.	Cliente/servidor; también ofrece PubSub [10].
Uso típico	Leer registros de inversores, medidores y PLC en campo [11].	Telemetría ligera hacia un sistema central o la nube [11].	Integración entre planta y TI: historiadores, MES, sistemas de terceros [11].
Formato de los datos	Registros y bobinas planos, sin significado propio [10].	Payload libre (p. ej. JSON) bajo un topic; no impone modelo [10].	Modelo de información con objetos, tipos y semántica [10][11].
Seguridad	Casi nula: sin cifrado ni autenticación propios [10].	Opcional, sobre el transporte (TLS, usuario y clave) [10].	Integrada: certificados, cifrado y firma de mensajes [10][11].

14 · Payload JSON del inversor y consultas JSONB

El inversor publicaría cada 5 minutos un mensaje con nueve campos (alarma aparece solo cuando hay falla).
Unidades: kW, V, A, Hz, °C y kWh.

```

{
  "device_id": 1,
  "ts": "2026-10-05T06:05:00-05:00",
  "p_ac_kw": 3.214,      // potencia AC, kW
  "v_dc_v": 612.4,      // voltaje DC, V
  "i_dc_a": 5.3,        // corriente DC, A
  "f_hz": 60.01,        // frecuencia de red, Hz
  "temp_inv_c": 41.2,    // temperatura del inversor, °C
  "e_dia_kwh": 12.8,     // energía acumulada del día, kWh
  "estado": "RUN"
} // (los comentarios // son solo explicativos; el JSON real no los lleva)

```

```

-- Consulta 1: operadores -> y ->> (lecturas con potencia mayor de 3 kW)
SELECT payload->'device_id' AS dispositivo, payload->>'ts' AS ts,
       (payload->>'p_ac_kw')::numeric AS p_ac_kw
FROM   stg_lectura_raw
WHERE  (payload->>'p_ac_kw')::numeric > 3
ORDER BY p_ac_kw DESC;

-- Consulta 2: operadores ? y @> (mensajes en estado RUN que traen alarma)
SELECT payload->>'device_id' AS dispositivo, payload->>'alarma' AS alarma
FROM   stg_lectura_raw
WHERE  payload ? 'alarma'
AND    payload @> '{"estado": "RUN"}';

```

I5 · Volumen de datos, particionamiento y BRIN

10 dispositivos × 5 variables × 1 lectura/min × 60 × 24 × 30 días = **2 160 000 filas por mes** (≈ 26 millones al año), suponiendo una fila por variable y mes de 30 días.

El **particionamiento por rango** sobre `ts` divide la tabla en una partición por mes: las consultas por fechas solo recorren las particiones relevantes y retirar un mes antiguo es cuestión de desprender o eliminar una partición. El **índice BRIN** guarda solo el mínimo y el máximo de cada grupo de bloques; como lectura solo recibe datos nuevos con `ts` creciente, ese resumen es muy preciso y el índice resulta mucho más pequeño que un B-tree [12]. **TimescaleDB** es una extensión de PostgreSQL que automatiza el particionamiento por tiempo mediante hypertables [13].

Bloque 4 · Control de versiones

V1 · Git, GitHub y credenciales

Git es el sistema de control de versiones que corre en el computador; **GitHub** es el servicio en la nube que aloja repositorios Git y añade colaboración [14]. Un **repository** es la carpeta del proyecto con todo su historial. Un **commit** es una instantánea guardada con un mensaje. Un **branch** es una línea paralela de trabajo [14]. Un **pull request** es la solicitud, propia de GitHub, para revisar y fusionar una rama en `main` [14]. El **.gitignore** lista los archivos que Git no debe rastrear.

Las credenciales nunca se suben porque el repositorio es público y lo que entra al historial queda allí aunque se borre después. Se evita con variables de entorno, un `.env` excluido en `.gitignore` y un `.env.example` sin valores [15]. Si se filtra una clave, se cambia de inmediato.

Parte B · Práctica guiada: primer pipeline de ingesta

Código completo en el repositorio (etl/, sql/). Cada paso incluye su explicación y la captura de evidencia.

Paso 1 · Simulador IoT

// El script etl/simulador.py genera lecturas de dos dispositivos (dispositivo_id 1 y 2) cada 5 minutos durante 12 horas: 144 instantes × 2 = 288 líneas en data/lecturas.jsonl. Se renombraron las claves a dispositivo_id y timestamps y se añadieron los campos v_ac (voltaje AC, en V) e ingresado_en (marca de tiempo de ingreso del mensaje).

```
PS C:\solardb-echeverri-quintero> & C:\Users\<user>\AppData\Local\Programs\Python\Python314\python.exe c:/solardb-echeverri-quintero/etl/simulador.py
PS C:\solardb-echeverri-quintero>
```

```
data > () lecturas.jsonl
1 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:00:00-05:00", "p_ac": 3.445, "v_ac": 199.4, "irradiancia": 874.5, "temp_modulo": 42.4, "ingresado_en": "2026-10-05T06:00:00-05:00"}]
2 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:00:00-05:00", "p_ac": 3.83, "v_ac": 110.8, "irradiancia": 706.3, "temp_modulo": 47.3, "ingresado_en": "2026-10-05T06:00:00-05:00"}]
3 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:05:00-05:00", "p_ac": 2.238, "v_ac": 192.5, "irradiancia": 687.7, "temp_modulo": 33.0, "ingresado_en": "2026-10-05T06:05:00-05:00"}]
4 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:05:00-05:00", "p_ac": 1.887, "v_ac": 110.8, "irradiancia": 895.9, "temp_modulo": 48.6, "ingresado_en": "2026-10-05T06:05:00-05:00"}]
5 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:10:00-05:00", "p_ac": 3.29, "v_ac": 131.0, "irradiancia": 173.9, "temp_modulo": 28.8, "ingresado_en": "2026-10-05T06:10:00-05:00"}]
6 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:10:00-05:00", "p_ac": 1.134, "v_ac": 184.1, "irradiancia": 113.2, "temp_modulo": 41.5, "ingresado_en": "2026-10-05T06:10:00-05:00"}]
7 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:15:00-05:00", "p_ac": 2.564, "v_ac": 137.7, "irradiancia": 131.7, "temp_modulo": 53.8, "ingresado_en": "2026-10-05T06:15:00-05:00"}]
8 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:15:00-05:00", "p_ac": 0.535, "v_ac": 155.6, "irradiancia": 866.3, "temp_modulo": 39.2, "ingresado_en": "2026-10-05T06:15:00-05:00"}]
9 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:20:00-05:00", "p_ac": 2.685, "v_ac": 218.4, "irradiancia": 258.0, "temp_modulo": 23.9, "ingresado_en": "2026-10-05T06:20:00-05:00"}]
10 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:20:00-05:00", "p_ac": 2.551, "v_ac": 148.8, "irradiancia": 689.6, "temp_modulo": 57.5, "ingresado_en": "2026-10-05T06:20:00-05:00", "alarma": "GRID_FAULT"}]
11 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:25:00-05:00", "p_ac": 0.391, "v_ac": 220.9, "irradiancia": 417.9, "temp_modulo": 27.6, "ingresado_en": "2026-10-05T06:25:00-05:00"}]
12 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:25:00-05:00", "p_ac": 1.201, "v_ac": 115.1, "irradiancia": 688.3, "temp_modulo": 42.8, "ingresado_en": "2026-10-05T06:25:00-05:00"}]
13 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:30:00-05:00", "p_ac": 4.416, "v_ac": 226.9, "irradiancia": 344.9, "temp_modulo": 46.8, "ingresado_en": "2026-10-05T06:30:00-05:00", "alarma": "GRID_FAULT"}]
14 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:30:00-05:00", "p_ac": 2.282, "v_ac": 206.1, "irradiancia": 576.1, "temp_modulo": 38.9, "ingresado_en": "2026-10-05T06:30:00-05:00"}]
15 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:35:00-05:00", "p_ac": 0.823, "v_ac": 154.5, "irradiancia": 589.4, "temp_modulo": 36.2, "ingresado_en": "2026-10-05T06:35:00-05:00"}]
16 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:35:00-05:00", "p_ac": 2.468, "v_ac": 227.6, "irradiancia": 772.4, "temp_modulo": 44.4, "ingresado_en": "2026-10-05T06:35:00-05:00"}]
17 [{"dispositivo_id": 1, "timestamps": "2026-10-05T06:40:00-05:00", "p_ac": 4.788, "v_ac": 118.0, "irradiancia": 413.8, "temp_modulo": 25.2, "ingresado_en": "2026-10-05T06:40:00-05:00"}]
18 [{"dispositivo_id": 2, "timestamps": "2026-10-05T06:40:00-05:00", "p_ac": 3.466, "v_ac": 131.6, "irradiancia": 991.0, "temp_modulo": 58.5, "ingresado_en": "2026-10-05T06:40:00-05:00"}]
```

Paso 2 · Staging y tabla destino

sql/01_tablas.sql crea stg_lectura_raw y lectura_demo adaptando el esquema del enunciado a los campos del simulador (dispositivo_id, timestamps, v_ac, alarma e ingresado_en), y se añade etl_log. Para cargar el JSONL se eligió **Python con psycopg2** en lugar de \copy: cada línea es un JSON con comillas, comas y llaves que el formato de texto o CSV de \copy obliga a escapar con cuidado, mientras que psycopg2 la envía como parámetro jsonb, permite validar cada línea y cuenta las filas para la bitácora, todo desde un único comando.

Query Query History	
1	SELECT * FROM stg_lectura_raw LIMIT 5;
Data Output Messages Notifications	
Showing rows: 1 to 5 Page No: 1	
id [PK] bigint	payload jsonb
1	1297 {"p_ac": 1.6, "v_ac": 193.6, "timestamp": "2026-10-05T06:00:00-05:00", "irradiancia": 617.3, "temp_modulo": 47.0, "ingresado_en": "2026-10-05T06:00:00-05:00", "dispositivo_id": 1}
2	1298 {"p_ac": 1.485, "v_ac": 137.0, "timestamp": "2026-10-05T06:05:00-05:00", "irradiancia": 662.5, "temp_modulo": 45.3, "ingresado_en": "2026-10-05T06:05:00-05:00", "dispositivo_id": 1}
3	1299 {"p_ac": 2.281, "v_ac": 221.8, "timestamp": "2026-10-05T06:10:00-05:00", "irradiancia": 522.1, "temp_modulo": 34.2, "ingresado_en": "2026-10-05T06:10:00-05:00", "dispositivo_id": 1}
4	1300 {"p_ac": 4.45, "v_ac": 153.4, "timestamp": "2026-10-05T06:15:00-05:00", "irradiancia": 620.5, "temp_modulo": 58.4, "ingresado_en": "2026-10-05T06:15:00-05:00", "dispositivo_id": 1}
5	1301 {"p_ac": 4.328, "v_ac": 142.2, "timestamp": "2026-10-05T06:20:00-05:00", "irradiancia": 561.8, "temp_modulo": 41.6, "ingresado_en": "2026-10-05T06:20:00-05:00", "dispositivo_id": 1}

Paso 3 · Transformación y carga

La sentencia extrae cada atributo del payload con ->, lo convierte al tipo de su columna destino y lo inserta en lectura_demo. El WHERE descarta las filas incompletas o fuera de rango (potencia negativa o irradiancia fuera de 0–1500 W/m²) y ON CONFLICT (dispositivo_id, timestamps) DO NOTHING evita duplicados. Los campos v_ac, alarma e ingresado_en se almacenan como columnas propias.

Query	Query History
1	INSERT INTO lectura_demo (
2	dispositivo_id,
3	timestamps,
4	p_ac,
5	v_ac,
6	irradiancia,
7	temp_modulo,
8	alarma,
9	ingresado_en
10	)
11	SELECT
12	(payload->>'dispositivo_id')::INT,
13	(payload->>'timestamps')::TIMESTAMPTZ,
14	(payload->>'p_ac')::NUMERIC(10,3),
15	(payload->>'v_ac')::NUMERIC(5,1),
16	(payload->>'irradiancia')::NUMERIC(8,1),
17	(payload->>'temp_modulo')::NUMERIC(5,1),
18	payload->>'alarma',
19	COALESCE (
20	(payload->>'ingresado_en')::TIMESTAMPTZ,
21	NOW ()
22	)
23	FROM stg_lectura_raw
24	WHERE payload ? 'dispositivo_id'
25	AND payload ? 'timestamps'
26	AND COALESCE (
27	(payload->>'p_ac')::NUMERIC,
28	0
29	) >= 0
30	AND COALESCE (
31	(payload->>'irradiancia')::NUMERIC,
32	0
33	) BETWEEN 0 AND 1500
34	ON CONFLICT (dispositivo_id, timestamps)
35	DO NOTHING ;

Data Output	Messages	Notifications
INSERT 0 288		
Query returned successfully in 25 msec.		

Paso 4 · Prueba de idempotencia

Resultado: `SELECT COUNT(*) FROM lectura_demo;` devuelve **288** tras la primera ejecución y **288** tras la segunda. En la segunda corrida el staging se recarga con las mismas 288 líneas, pero cada par (dispositivo_id, timestamps) ya existe, por lo que `ON CONFLICT DO NOTHING` omite todas las filas y filas_cargadas queda en 0 [5]. Repetir el flujo no duplica datos: es idempotente.

Query Query History

```
1 SELECT COUNT(*) FROM lectura_demo;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count	bigint
1	288	

Successfully run. Total query runtime: 34 msec. 1 rows affected. x

Successfully run. Total query runtime: 37 msec. 1 rows affected. x

Successfully run. Total query runtime: 36 msec. 1 rows affected. x

Successfully run. Total query runtime: 39 msec. 1 rows affected. x

Successfully run. Total query runtime: 35 msec. 1 rows affected. x

Successfully run. Total query runtime: 39 msec. 1 rows affected. x

Paso 5 · Bitácora y automatización

etl_log se diseñó según E4: id de ejecución, proceso, inicio, fin, filas leídas, cargadas y rechazadas, estado y mensaje de error. etl/run_etl.py es el único punto de entrada (python etl/run_etl.py): ejecuta el flujo completo (carga a staging, transformación y registro en etl_log) y, si ocurre una falla, deja constancia del estado y del mensaje de error. Programación cada hora (sin implementarla):

```
# Linux / macOS (crontab -e): minuto 0 de cada hora
0 * * * * cd /ruta/solardb-echeverri-quintero && python etl/run_etl.py >> logs/etl.log 2>&1

REM Windows (Programador de tareas, desde la consola)
schtasks /create /sc hourly /tn "SolarDB_ETL" /tr "python C:\solardb-echeverri-quintero\etl\run_etl.py"
```

Query Query History

```
1 SELECT * FROM etl_log;
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No:

	id [PK] bigint	proceso character varying (100)	inicio timestamp with time zone	fin timestamp with time zone	filas_leidas integer	filas_cargadas integer	filas_rechazadas integer	estado character varying (20)	mensaje_error text
1	15	Ingesta_JSONL_Solar	2026-10-04 23:05:41.095793...	2026-10-04 23:05:41.165863...	288	0	0	EXITOSO	[null]
2	16	Ingesta_JSONL_Solar	2026-10-04 23:05:50.824908...	2026-10-04 23:05:50.892935...	288	0	0	EXITOSO	[null]

Paso 6 · Gobernanza aplicada

Se creó solar_lector con GRANT SELECT ON lectura_demo y sin ningún otro permiso (sql/03_roles.sql). Conectado con ese rol, el SELECT se ejecuta correctamente, mientras que el INSERT es rechazado con el error permission denied for table lectura_demo (SQLSTATE 42501): es el mínimo privilegio en acción.

The screenshot shows the SolarDB web interface. At the top, the user is logged in as 'solar_lector@SolarDB - Lector'. Below the navigation bar, the 'Query' tab is active, displaying the following SQL statement:

```
1 INSERT INTO lectura_demo (dispositivo_id, ts, p_ac) VALUES (1, NOW(), 2.5);
```

Below the query editor, the 'Data Output' tab is selected, showing an error message:

```
ERROR: permission denied for table lectura_demo
SQL state: 42501
```

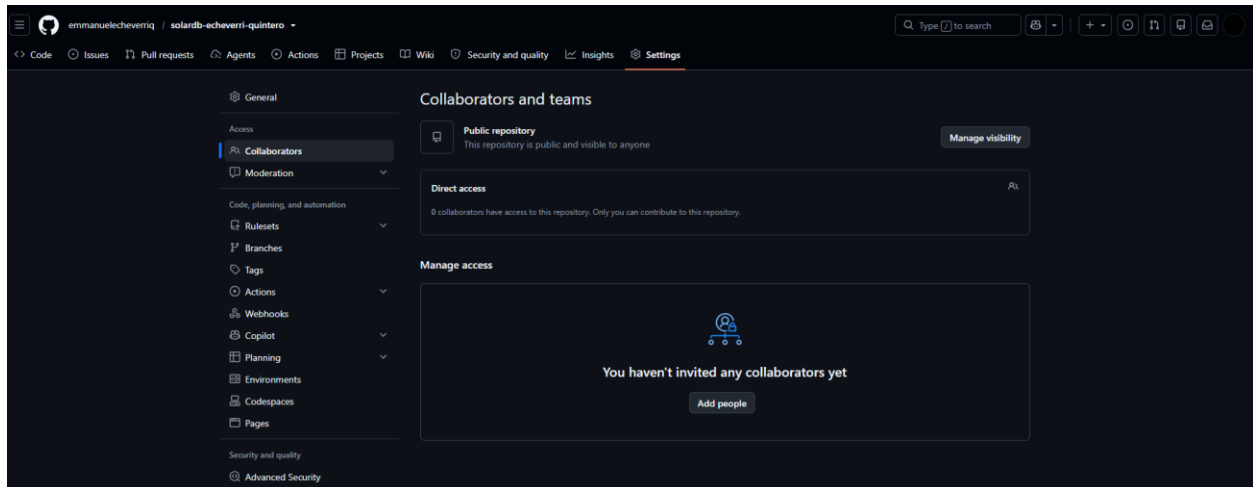
Parte C · Repositorio en GitHub

Enlace al repositorio (público, nombre solardb-echeverri-quintero):

<https://github.com/emmanuelecheverri/solardb-echeverri-quintero>

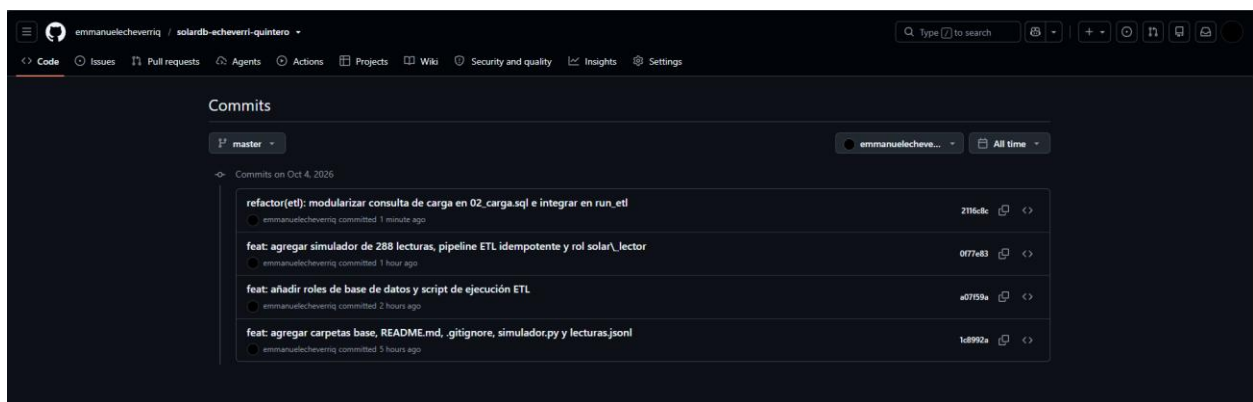
Estructura del repositorio: README.md, .gitignore, docs/, data/lecturas.jsonl, etl/simulador.py, etl/run_etl.py, sql/01_tablas.sql, sql/02_carga.sql y sql/03_roles.sql. El .gitignore excluye .env, y las credenciales se leen solo de variables de entorno.

Captura de la sección Collaborators



No hay colaboradores (Trabajo individual)

Captura del historial de commits



Referencias

- [1] DAMA International. (2017). DAMA-DMBOK: Data Management Body of Knowledge (2.ª ed.). Technics Publications.
- [2] Data Crossroads. (2024, 31 de diciembre). Governance vs. management: clarifying the divide. <https://datacrossroads.nl/2024/12/31/governance-vs-management-clarifying-the-divide>
- [3] Lights On Data. Data quality and data governance. <https://www.lightsondata.com/data-quality-and-data-governance/>
- [4] Superintendencia de Industria y Comercio. Concepto 15-283428, sobre la definición de dato personal (art. 3 de la Ley 1581 de 2012). https://sedeelectronica.sic.gov.co/sites/default/files/normatividad/Concepto_15-283428.pdf
- [5] PostgreSQL Global Development Group. INSERT (cláusula ON CONFLICT). Documentación de PostgreSQL. <https://www.postgresql.org/docs/current/sql-insert.html>
- [6] Citus Data. pg_cron: job scheduler for PostgreSQL. https://github.com/citusdata/pg_cron
- [7] Apache Software Foundation. Apache Airflow Documentation. <https://airflow.apache.org/docs/>
- [8] MotherDuck. Apache Airflow (glosario). <https://motherduck.com/glossary/apache-airflow/>
- [9] HiveMQ. Publish/Subscribe Architecture. <https://docs.hivemq.com/hivemq-platform/connect/publish-subscribe-architecture.html>
- [10] RF Wireless World. OPC UA vs Modbus vs MQTT. <https://www.rfwireless-world.com/terminology/networking-basics/opc-ua-vs-modbus-mqtt>
- [11] Industrial Shields. Modbus RTU vs Modbus TCP vs OPC UA: choosing the right plant communications architecture. <https://www.industrialshields.com/blog/prototype-to-plant-34/modbus-rtu-tcp-opc-ua-709>
- [12] StackHarbor. BRIN indexes for very large PostgreSQL tables. <https://stackharbor.com/en/knowledge-base/pg-brin-indexes-large-tables/>
- [13] Timescale. TimescaleDB (repositorio y documentación). <https://github.com/timescale/timescaledb>
- [14] ABC Steps. GitHub for Beginners: creating your first repository and pull request. <https://abcsteps.com/blog/github-for-beginners-creating-your-first-repository-and-pull-request>
- [15] Promptz. Git best practices (seguridad: no subir secretos, usar variables de entorno). <https://www.promptz.dev/steering/kiro-best-practices-steering-git-best-practices>